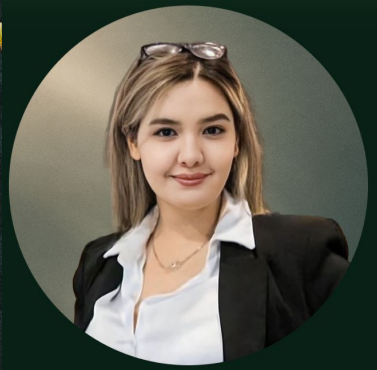
Deep Learning & AI in Sports

twelve



Dr. Pegah Rahimian

Dr. Pegah Rahimian is a football data scientist at Twelve Football and a post-doctoral researcher at Uppsala University. She holds a PhD in Football Data Science and has applied AI to derive in-depth analyses from football data, influencing various aspects of the sport.

October 2025

Mondays & Thursdays:
17:00 - 19:00 CET

Professor David Sumpter

Professor David Sumpter is a professor of applied mathematics at Uppsala University and the author of "Soccermatics," a book that explores the application of mathematical modeling to football. He co-founded Twelve Football and has worked with major clubs and national federations worldwide.

Course Roadmap

The key is to move from **structure** → **interactions** → **valuation** → **communication**,

📊 **Session 1:** Tracking → Team Shapes, defensive lines & Formation Clusters

🧩 **Session 2:** Playing-Style & Role Representations (clustering/embeddings)

🌐 **Session 3:** Graph Neural Networks for Pass Risk & Reward, Pitch Control

⚡ **Session 4:** Action Valuation I — Expected Threat (xT), EPV & VAEP

🎯 **Session 5:** Action Valuation II — Plus–Minus & Reinforcement Learning

🔍 **Session 6:** Interpretability & Explainable ML in Football

💬 **Session 7:** Communicating With LLM Assistants (coach-ready insights)

🏟️ **Session 8:** Project Studio & Demos

Module 3:
Graph Neural Networks (GNNs) for Football Tracking Data

1. Dealing with Unstructured Tracking Data in Football
2. Graph Neural Networks for Football Tracking Data
3. Implementation

1. Dealing with Unstructured Tracking Data in Football

Problem:

Tracking data captures continuous spatiotemporal trajectories of all players and the ball — not in any fixed order.

Issue:

Most machine-learning models (e.g., MLPs, CNNs, RNNs) expect **tabular or sequential data**, where features are ordered consistently (e.g., player 1, 2, 3,…).

Result:

We must artificially impose an *ordering or alignment* on players before learning patterns.

Goal:

Find a representation that is

- **Permutation-invariant** (order doesn't matter)
- **Robust to missing players**
- **Computationally efficient** for real-time inference

Method 1: Formation Alignment Templates

Key Idea (Lucey et al. 2013 [1])

Align players to a **formation template** (e.g., 4-3-3, 3-5-2) so that features appear in a consistent spatial order.
Each player's trajectory is assigned to a fixed "role position" slot.

Advantages:

- Simple to implement once formation is known.
- Allows frame-by-frame feature matrices for standard ML models.

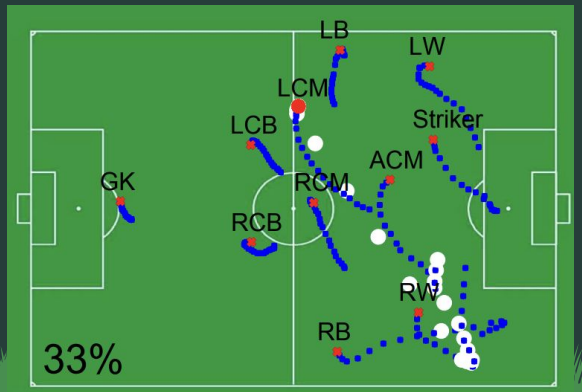
Drawbacks:

- ⚠️ Formation-dependent — a "role 10" in 4-3-3 ≠ "role 10" in 3-5-2.
- ⚠️ Fails when player counts differ (red cards, substitutions).
- ⚠️ Requires fast pre-alignment before inference → latency issue.

Summary:

✅ Ordering enforced by tactical template
❌ Not generalizable across teams or dynamic shapes

Fig. 2. An example of the tracking data for both player and ball. Player positions are shown with their assigned role (discussed in Section IV.A.). In this work, we focussed on a team which used a 4-3-3 formation.

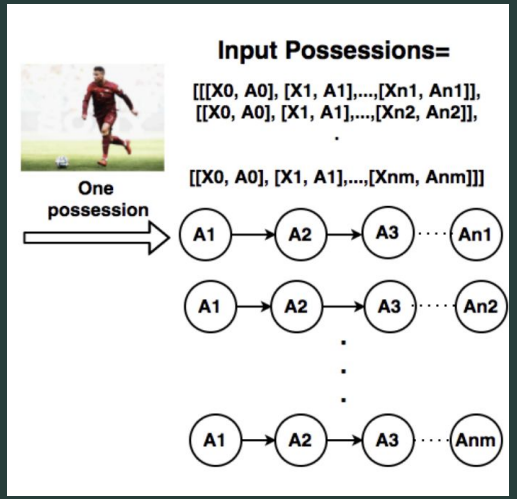


Method 2: Permutation-Invariant Sorting Schemes

Key Idea (Nazanin et al. 2018 [2]; Rahmian et al. 2021, 2022 [3,4])
Order players **relative to an anchor** (the ball or ball carrier).
The anchor trajectory is placed first; other players are sorted by their **distance to the anchor**.

Procedure:

1. Select anchor → ball or carrier.
2. Compute distances of all players to anchor.
3. Sort players ascending by distance.
4. Concatenate ordered trajectories as model input.



Benefits:

- Permutation invariance within team.
- Works with variable formations and roles.
  Consistent reference frame (ball/anchor-centric).

Drawbacks:

- Still imposes an *artificial order*.
  Sensitive to anchor detection errors.
- Sorting overhead per frame.

Method 3: *Image-Based Representations*

Key Idea (Fernández et al. (2020) [5]; Brefeld et al. [6] , Rahimian et al. (2022) [7])
Convert tracking frames into **image-like spatial grids**, then apply **Convolutional Neural Networks (CNNs)** to learn probabilistic *pitch-control surfaces*.

Fine-grained partitioning of the pitch:

Contains 105×68 = 7140 disjoint zones

Benefits:

- Removes explicit ordering/alignment.
- Leverages mature CNN architectures.
- Enables dense spatial reasoning.

Drawbacks:

- ⚠️ Tracking data → low-dimensional → converting to images makes it **high-dimensional and sparse**.
- ⚠️ Computationally heavy for real-time use.
- ⚠️ Loses fine-grained player identities and relations.
- ⚠️ Difficult to handle missing players.

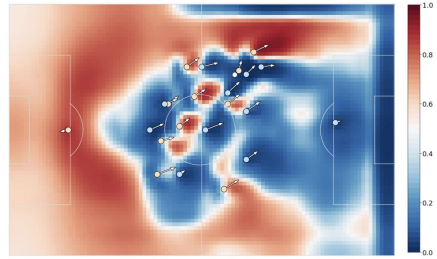
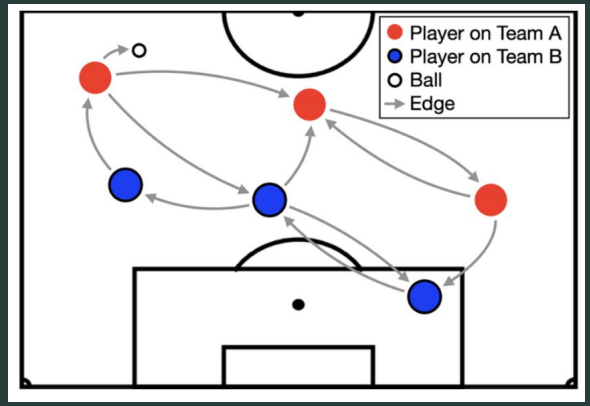


Fig. 3: Pass probability surface for a given game situation. Yellow and blue circles represent players' locations on the attacking and defending team, respectively, and the arrows represent the velocity vector for each player. The white circle represents the ball location.

Method 4: Graph-Based Representations

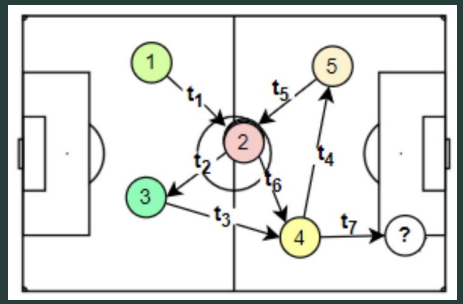
Key Idea (Power et al. 2021 [8], Dick & Brefeld 2022 [9], Rahimian et al. 2023 [10, 11])

Model players + ball as **nodes in a fully-connected graph**.
Edges represent **interactions** (e.g., distance, velocity difference, passing angle).
Apply **Graph Neural Networks (GNNs)** to learn relational patterns.



Advantages:

- 🧩 Permutation-invariant by design (node order irrelevant).
- ⚽ Handles variable numbers of players per frame.
- 🔗 Captures local and global context simultaneously.



Drawbacks:

- Requires defining meaningful edge features.
- Computationally more complex than pure MLPs.

Comparative Overview

Approach	Representation	Handles Variable # Players	Order-Free	Real-Time Feasible	Main Drawback
Formation Template [1]	Fixed role alignment	❌	❌	⚠️ Limited	Formation-specific
Permutation Sorting [2,3,4]	Distance-based order	⚙️ Partial	⚠️ Weak	✅	Still sequential
Image Representation [5,6,7]	Spatial grid (CNN)	✅	✅	❌	High dimensional / sparse
Graph Neural Network [8,9,10,11]	**Nodes + Edges**	✅	✅	✅	Edge feature design

2. Graph Neural Networks for Football Tracking Data

Why Graphs Fit Naturally with Football

Football is relational.
Every player's behavior depends on others — teammates, opponents, and the ball.

This is **not** a fixed table of numbers, but a **dynamic web of interactions**.

Concept	Football Analogy
Node	Player or ball
Edge	Interaction between two entities (e.g., marking, passing lane, pressure)
Edge feature	Relative distance, angle, velocity difference
Graph	Snapshot of the pitch (a frame of play)

Why graphs?

⚽ Football = system of relations, not isolated points

🧩 GNNs handle variable numbers of nodes (10v10, 11v11, etc.)

🔄 Permutation-invariant — node order doesn't matter

💡 Naturally captures team structure, space occupation, and dynamics

Constructing the Graph from Tracking Data

Step 1: **Define Nodes**

- Each **player** = 1 node
- The **ball** = 1 special node

Step 2: **Define Node Features:**

Feature Type	Example
Spatial	(x_norm, y_norm) position
Motion	(vx, vy) velocity
Identity	Team flag, is_ball, is_carrier
Context	Distance to goal, pitch zone

Constructing the Graph from Tracking Data

Step 3: Define Edges

Edge Feature	Meaning
Δx, Δy	Relative displacement
Distance	Proximity between entities
Angle	Direction from one player to another
Same-Team Flag	Teammate vs opponent
Closing Speed	Velocity projection toward each other

Result:
Each frame becomes a **fully connected graph** (22 players + 1 ball = 23 nodes, 23×22 edges).

🧩 This structure is invariant to player numbering and adaptable to missing players.

GNN Fundamentals: Message Passing

Core Idea:
Nodes update their state by *exchanging messages* with connected nodes.

Mathematically:

$$h_i^{(k+1)} = \text{UPDATE}\left(h_i^{(k)}, \text{AGGREGATE}_{j \in \mathcal{N}(i)} f(h_i^{(k)}, h_j^{(k)}, e_{ij})\right)$$

Intuition in football:

Each player updates their "awareness" based on signals from nearby players and the ball.

Step-by-step analogy:

1. Each node observes its **neighbors** (message passing).
2. It aggregates those messages (mean, sum, attention).
3. It updates its own feature embedding (new understanding of the situation).
4. Repeat for multiple layers (information propagates globally).

Popular GNN Variants in Sports Analytics

Model	Key Mechanism	Football Analogy	Pros	Cons
Graph Convolutional Network (GCN)	Simple mean aggregation	Each player “averages” info from neighbors	Easy to implement	May oversmooth features
GraphSAGE	Sample and aggregate neighbors	Player learns localized pattern from close teammates	Scalable	May miss global context
Graph Attention Network (GAT)	Learns attention weights	Player focuses more on *important* neighbors (e.g., nearest opponents)	Interpretable attention weights	Slightly heavier compute
Graph Recurrent Neural Network (GRNN)	Temporal message passing	Learns evolving context across frames	Captures temporal dynamics	Complex and slow

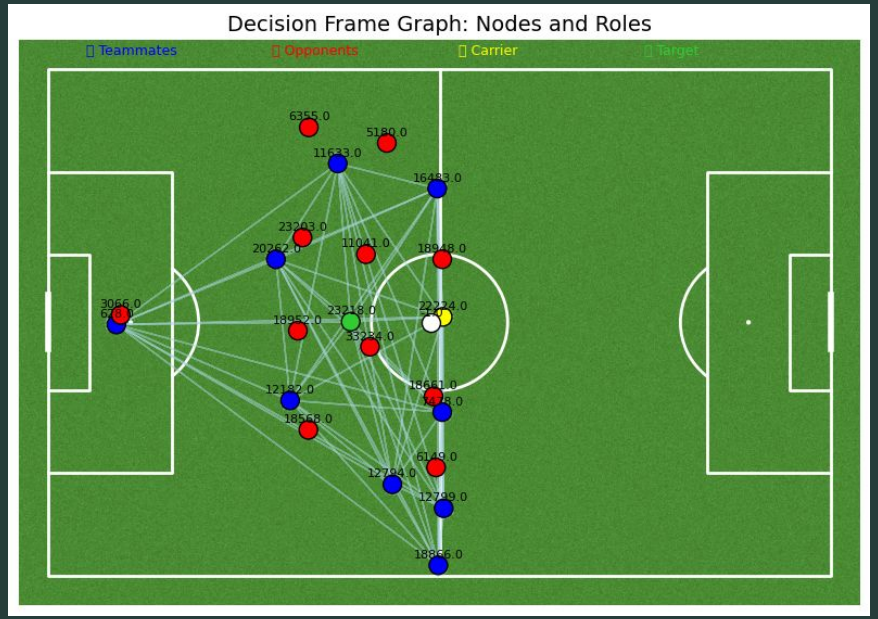
Example 1: Modeling Pass Decisions

Task: Predict *who* the ball carrier will pass to (receiver prediction).

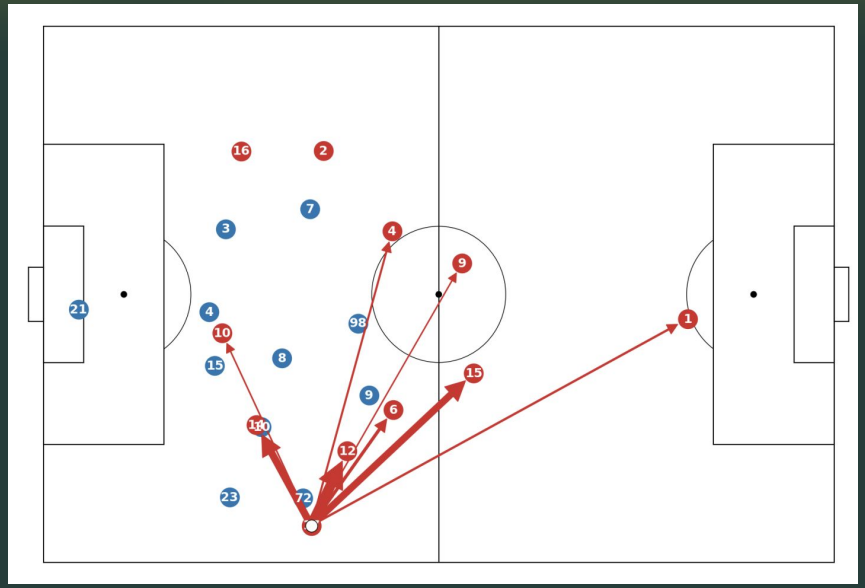
Graph setup:

- Nodes: 23 entities (22 players + ball)
- Edges: Fully connected, with geometric & dynamic features
- Label: Index of the *actual receiver* in the frame

GNN output:
Each node outputs a scalar “pass likelihood score”.
The node with the highest score = predicted receiver.



Example 1: Modeling Pass Decisions



Interpretation:

The GNN learns to infer passing intent by reading relational cues — spacing, pressure, angles, and teammate availability.

Example 2: Modeling Pass Success (xPass)

Task: Predict the probability that a given pass will succeed.

Input: Graph of the moment *just before the pass*.
Label: 1 if the pass succeeded, 0 otherwise.

Why it works:

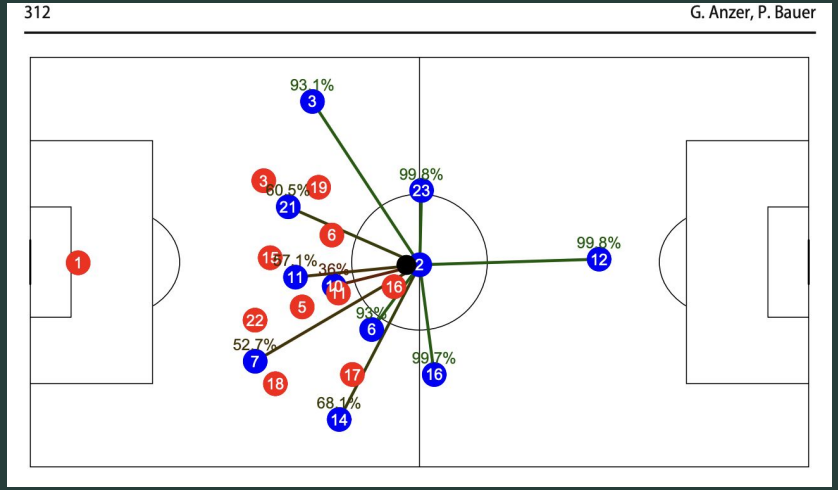
- The GNN considers **spatial density** (defenders nearby).
- Accounts for **angles and velocities** (pressure, risk).
- Uses **team context** (ball support, spacing).

Interpretation of output:

$$\text{xPass} = P(\text{successful pass} \mid \text{positions, motions, context})$$

Use case:

- **Quantify passing difficulty — not execution quality.**
- **A failed pass with high xPass = execution error.**
- **A successful pass with low xPass = exceptional skill.**



Anzer and Bauer 2022 [12]

3. Implementation

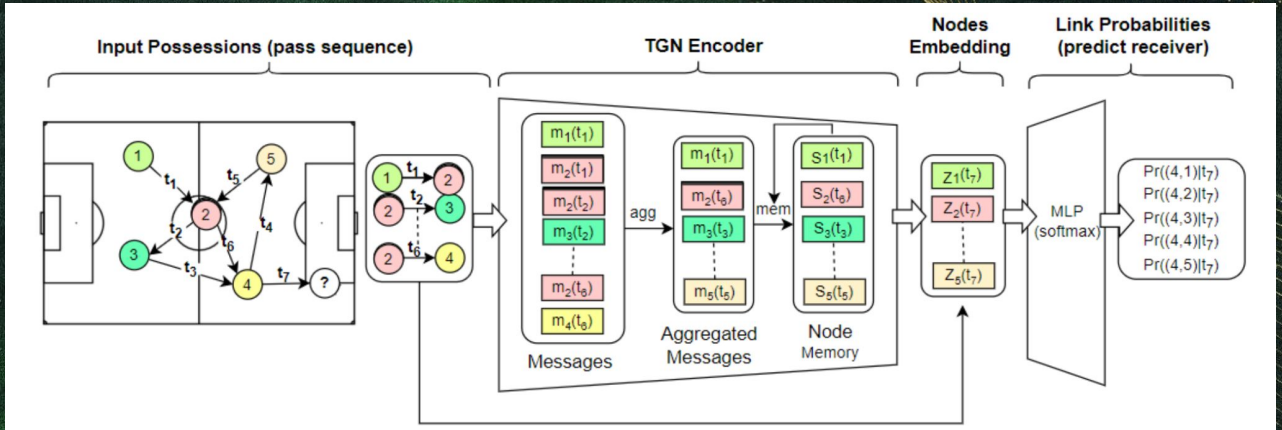


Fig. 1: TGN architecture for outcome prediction.

What are we predicting?

xReceiver — Who will receive the pass?

- **Type:** node classification (among teammates on the pitch)
- **Input:** one **decision frame** (just before the pass) → graph of 22 players + ball
- **Label:** index of the **actual receiver** (from event data)

Why it matters: anticipates passing choice → helps analyze decision-making, pressure, and options

xPass — Will the pass succeed?

- **Type:** graph-level binary classification (success/fail)
- **Input:** same decision frame (carrier + context)
- **Label:** 1 if the pass was completed, 0 otherwise

Interpretation: "probability this *actual attempted* pass will be completed"

Why it matters: separates *decision difficulty* from *execution quality*

xThreat (xT) — How much will the pass increase scoring chances?

- **Type:** graph-level binary classification (goal or not)
- **Input:** decision frame + target location (from event data)
- **Label:** 1 if a pass lead to a shot or goal within the next 10 seconds, otherwise 0.

Why it matters: quantifies *value creation* beyond completion — risk-reward, progression, danger

Implementation Overview (378 PL matches)

Goal: build a fast, reliable pipeline you can run end-to-end:

raw tracking/events → graphs → datasets → models → evaluation → visuals

1) 📦 Data intake & normalization

- **Tracking (10 fps)** normalized to **left→right**, meters or [0–110]×[0–68]
- Add **ball-in-play** mask, merge **team/player meta**
- Ensure **period**, **match_id**, **team_id** are consistent across sources
- Keep **22 players + 1 ball** frames when possible

Outputs per match: `processed/{match_id}.parquet`

Implementation Overview (378 PL matches)

2) Decision frames (labels come from events)

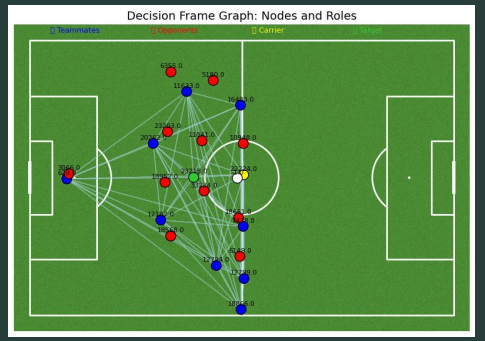
- From **dynamic events**:
  - Filter `event_type == "player_possession"`
  - For possessions ending in **pass/shot/loss**, compute a **decision frame**:
    - `decision_frame = max(frame_start, frame_end - safety_frames)` (e.g., safety=2)
- Attach **labels**:
  - *`player_in_possession_id`*, *`player_targeted_id`*, *`pass_outcome`*, *`lead_to_shot/goal`*
- (Optional) infer carrier if missing: nearest teammate to **ball** at decision frame

Outputs per match: `graph_inputs_decisions/{match_id}_decisions.parquet`

Implementation Overview (378 PL matches)

3) Graph construction (per decision frame)

- **Nodes (23):** 22 players + ball
  - Features: `[x, y, vx, vy, is_ball, team_flag (teammate/opponent), is_carrier, ahead_flag]`
- **Edges (fully connected, directed, no self-loops):**
  - Features: `Δx, Δy, distance, angle, same_team, closing_speed`
- **Targets:**
  - **xReceiver:** node index of `player_targeted_id` (teammates only)
  - **xPass:** `pass_outcome` → 1/0
  - **xThreat:** `xT(end_cell) - xT(start_cell)` (precompute xT grid on the same coordinate system)



Implementation Overview (378 PL matches)

4) Dataset building (all matches)

- **Iterate over 378 matches**:
  - Read decisions parquet → group by `(match_id, period, frame)`
  - Build graphs → **skip** scenes with missing target or bad geometry
- **Persist**:
  - `graphs_all.npz` (packed arrays) or one `*.pt` per scene
  - Keep **metadata**: `(match_id, period, frame, carrier_id, target_id, pass_outcome)`

Scale guidance:

- ~1.5–2.5k passes per team/season → **80k–120k** decision frames across 378 matches (order-of-magnitude).

Implementation Overview (378 PL matches)

5) Train/val/test split (match-wise)

- **Never** split within the same match (avoid leakage)
- Example:
  - Train: 70% matches
  - Val: 15% matches
  - Test: 15% matches
- Keep class balance in mind:
  - **xReceiver:** many negatives per node — balance via masking/sampling
  - **xPass:** typically ~70–85% success → use **class weighting** or **focal loss**
  - **xThreat:** skewed with many near-zero values → use **Huber loss** or **quantile bins** for metrics

```
Epoch 01: Loss = 25.6259, Validation Accuracy = 57.66%
Epoch 02: Loss = 21.1595, Validation Accuracy = 60.79%
Epoch 03: Loss = 20.2767, Validation Accuracy = 62.89%
Epoch 04: Loss = 19.7011, Validation Accuracy = 64.58%
Epoch 05: Loss = 19.3470, Validation Accuracy = 64.96%
Epoch 06: Loss = 19.0329, Validation Accuracy = 66.18%
Epoch 07: Loss = 18.8458, Validation Accuracy = 66.39%
Epoch 08: Loss = 18.7052, Validation Accuracy = 67.47%
Epoch 09: Loss = 18.5923, Validation Accuracy = 67.25%
Epoch 10: Loss = 18.4134, Validation Accuracy = 67.06%
```

Implementation Overview (378 PL matches)

6) Models (choose one or mix)

A) xReceiver (node classifier)

- **Architecture:** GraphSAGE / GAT → node logits → **masked softmax over candidate receivers**
- **Loss:** cross-entropy (only over **teammate, not ball, not carrier**)
- **Metrics:** Top-1 / Top-3 accuracy, MRR (mean reciprocal rank)

B) xPass (graph classifier)

- **Architecture:** GraphSAGE / GAT → **global mean/max pool** → MLP → sigmoid
- **Loss:** BCE (with class weights), report **AUC**, **Brier score**, **ECE** (calibration)

C) xThreat (graph regressor)

- **Architecture:** same as xPass, last layer **linear**
- **Loss:** Huber (robust), report MAE / RMSE / Spearman ρ
- **Interpretation:** bigger ΔxT = more value creation

Implementation Overview (378 PL matches)

7) ⚙️ Training recipe (practical)

- **Node/edge sanitization:** replace NaNs/inf with zeros; clip features (e.g., vx, vy)
- **Batching:** PyG `DataLoader(batch_size=32–128)`; pad nothing (graphs are variable by default)
- **Optim:** Adam (lr=1e-3), cosine decay or plateau scheduler
- **Regularization:** dropout 0.1–0.3; L2=1e-4
- **Monitoring:** early stopping on **val metric** (Top-1 for xReceiver, AUC for xPass and xT)
- **Logging:** loss curves + confusion/concordance charts; calibration plots (xPass)

Application: Passing performance

Column	Meaning
`mean_xpass`	The model's estimate of how safe a player's passes *should* be
`actual_pass_accuracy`	How often their passes were actually completed
`overperformance`	+ve → player performs better than expected; -ve → underperforms model's expectation

Overperforming Players

player_name	team_shortname	total_passes	mean_xpass	actual_pass_accuracy	overperformance
G. Edmundson	Ipswich Town	1	0.630429	1.0	0.369571
Vitor Reis	Manchester City	3	0.687855	1.0	0.312145
J. Meghoma	Brentford FC	2	0.692728	1.0	0.307272
S. McTominay	Manchester U	7	0.693006	1.0	0.306994
C. Echeverri	Manchester City	3	0.694269	1.0	0.305731
Chiquinho	Wolverhampton	3	0.694401	1.0	0.305599
F. Onyeka	Brentford FC	6	0.705873	1.0	0.294127
M. Amougou	Chelsea	8	0.708314	1.0	0.291686
J. Danns	Liverpool	1	0.719671	1.0	0.280329
W. Alves	Leicester	1	0.726662	1.0	0.273338

Underperforming Players

player_name	team_shortname	total_passes	mean_xpass	actual_pass_accuracy	overperformance
C. Kporha	Crystal Palace	2	0.721246	0.571429	-0.149817
R. Dixon	Everton	16	0.655460	0.500000	-0.155460
Carlos Vinícius	Fulham	4	0.664871	0.500000	-0.164871
Youssef Chermiti	Everton	7	0.674544	0.500000	-0.174544
Z. Silcott-Duberry	Bournemouth	2	0.684473	0.500000	-0.184473
H. Armstrong	Everton	14	0.668430	0.428571	-0.239858
Gustavo Nunes	Brentford FC	3	0.643740	0.333333	-0.310407
Luiz Felipe	Nottingham	1	0.600277	0.000000	-0.600277
T. King	Wolverhampton	1	0.699816	0.000000	-0.699816
W. Lankshear	Tottenham	1	0.712109	0.000000	-0.712109

Key Takeaways

- Football is **relational** → graphs are the *natural* representation
- **xReceiver** (choice), **xPass** (difficulty), **xThreat** (value) cover the decision-execution-impact chain
- With **clean decision frames** and **sensible node/edge features**, even simple GNNs (GraphSAGE/GAT) learn powerful patterns
- Robust pipelines (per-match processing, caching, sanitization) make **378 matches** totally tractable

References

[1] Lucey et al., "Large-Scale Analysis of Formations in Soccer,"International Conference on Digital Image Computing: Techniques and Applications (DICTA)", 2013

[2] Mehrasa et al. Deep Learning of Player Trajectory Representations for Team Activity Analysis, "MIT Sloan Sports Analytics Conference", 2018

[3] Rahimian et al. Towards optimized actions in critical situations of soccer games with deep reinforcement learning , "IEEE 8th International Conference on Data Science and Advanced Analytics (DSAA)", 2021

[4] Rahimian et al. A data-driven approach to assist offensive and defensive players in optimal decision making, "International Journal of Sports Science & Coaching", 2024

[5] Fernandez and Bornn, SoccerMap: A Deep Learning Architecture for Visually-Interpretable Analysis in Soccer, "ECML PKDD", 2020

[6] Brefeld et al, Probabilistic movement models and zones of control, "Machine Learning", 2018

[7] Rahimian et al, Beyond action valuation: A deep reinforcement learning framework for optimizing player decisions in soccer, "MIT Sloan Sports Analytics Conference", 2022

[8] Power et al., Making Offensive Play Predictable - Using a Graph Convolutional Network to Understand Defensive Performance in Soccer, "MIT Sloan Sports Analytics Conference", 2021

[9] Dick and Brefeld, Action rate models for predicting actions in soccer, "AStA Advances in Statistical Analysis", 2023

[10] Rahimian et al, Pass Receiver and Outcome Prediction in Soccer Using Temporal Graph Networks, "Machine Learning and Data Mining for Sports Analytics (MLSA)", 2023

[11] Rahimian et al., In-game soccer outcome prediction with offline reinforcement learning, "Machine Learning", 2024

[12] Anzer and Bauer, Expected passes Determining the difficulty of a pass in football (soccer) using spatio-temporal data, "Data Mining and Knowledge Discovery", 2022